

robot Cog. Rather than attempt to teach it everything, Brooks will let Cog find its own way in the world and learn on its own. This way, Cog would be able to respond to unexpected happenings in a more human-like manner—stumbling at first, but slowly learning from its mistakes.

It is difficult to say which approach will finally create the first truly intelligent machine, but the second approach might just win. The software of the future would be able to modify itself to behave the way users want it to. What's more, it would do this and still survive the big bad world it lives in.

It's Scary Out There

Life was easy for programmers when their end users were just computer professionals. There were less of them, and this meant fewer letters asking for new enhancements. Today, though, the number of users is staggering—including not just professionals but the layperson as well.

Add to this the complexity of today's systems: a company that used to have its employees connected to a single mainframe server, today, has a hornet's nest of desktop PCs and servers throughout its network.

And the cherry on the cake is the resources that need consideration—hardware resources, system load, network bandwidth, security and many more. Software needs to be able to trade off some of these resources in order to meet its goals.

Thus far, software has lived in a Utopia where every situation is planned for and every type of scenario is anticipated. The bitter truth is that programmers can't write applications that will tackle situations they can't foresee, and can't anticipate the new and creative ways in which users can cause software to misbehave.

In the real world, then, we need software that can react to and plan for sudden changes in its environment. Software that can alter itself to meet users' demands better, and can do so even in the face of chaos like failing hardware. The real world needs Adaptive Software.

Er... What Software?

In 1997, DARPA (Defense Advanced Research Project Agency)—an agency of the United States Department of Defense—defined Self-Adaptive Software as software that “evaluates its own behaviour and changes its behaviour when the evaluation indicates that it is not accomplishing what it is intended to do, or that better functionality or performance is possible”.

In nature's model of Life, the Universe and Everything, creatures have responded to changes in their environment by evolving to suit it; the intention is the same in the case of adaptive software—it alters itself according to its environment. The “environment” for a software application is complex, just like nature. There is hardware to live in, the ‘society’ of the operating system, and other software and humans to interact with. But how does software know what to do to adapt to this environment?

Put simply, Adaptive or Self-Adaptive Software “knows itself.” This means that in addition

The real world needs Adaptive Software—which can alter itself to meet users' demands better, and can do so even in the face of chaos like failing hardware

to knowing *what* it has to do, the software also knows its own structure—*how* it does what it does—and the many different ways to achieve its purpose. This way, if it isn't doing its job well, the software will automatically alter itself to do it better.

What Would You Do?

Let's take a look at how we interact with our environments, and how adaptive software follow the same behaviour in their own environments.

What if you were on your way to the cinema hall and your car broke down? Simple, really. Get out, take a cab, and move to your destination. Today's software is the equivalent of someone who'd sit in the car and cry about being unable to proceed—decidedly unpleasant company. Adaptive software, however, will be able to keep going despite such hardware failures. Like us, it would try to find a different method before giving up altogether. This is called *System Level Adaptivity*.

Suppose, now, that you are back home, still wondering how to go to that movie theatre. In Mumbai, you could take the train, which would be fast, but crowded and smelly. You could also take the bus, but that's just uncomfortable. Taxis are too expensive, and you have the sudden premonition that your car is going to break down.

Now that you've spent so much time mulling over what to do, you realise it's really late and that you should be getting on. The best way, then, is the train. In quite a similar manner, adaptive software looks at the status of the system and decides the best approach out of many possible approaches. This is called *Algorithm Level Adaptivity*.

Now picture yourself budgeting for that shiny new PC you're going to build. What you could do is find out the exact price of everything right down to the xx99.95 plus taxes. Accurate down to the last paisa, no doubt, but it would take you ages to figure out what you need.

What you could do instead is quickly just find out the approximate prices of everything and be on your way. You've traded in accuracy for time. Adaptive software will be able to decide how to make its calculations depending on how much accuracy is really needed for a purpose. If it isn't a big concern, it would use a less accurate method to save time and system resources. This is called *Application Level Adaptivity*.

The best adaptive software will be adaptable at all these three levels.

So How Do They Work?

It's all very well to talk about what adaptive software is capable of, but we now need more than just science fiction. We need to know just how this “revolution” proposes to deliver what it promises.

We expect software to behave like humans; the first step, then, is to have them thinking like humans. One of the ways in which adaptive software works is much like how we work. Instead of just running into a task, it plans its approach, executes it and then evaluates it. If the evaluation shows any negative results, it re-plans the